

WELFARE CONNECT

AI-Based Multilingual Chatbot for Welfare Scheme Awareness

📄 Multilingual

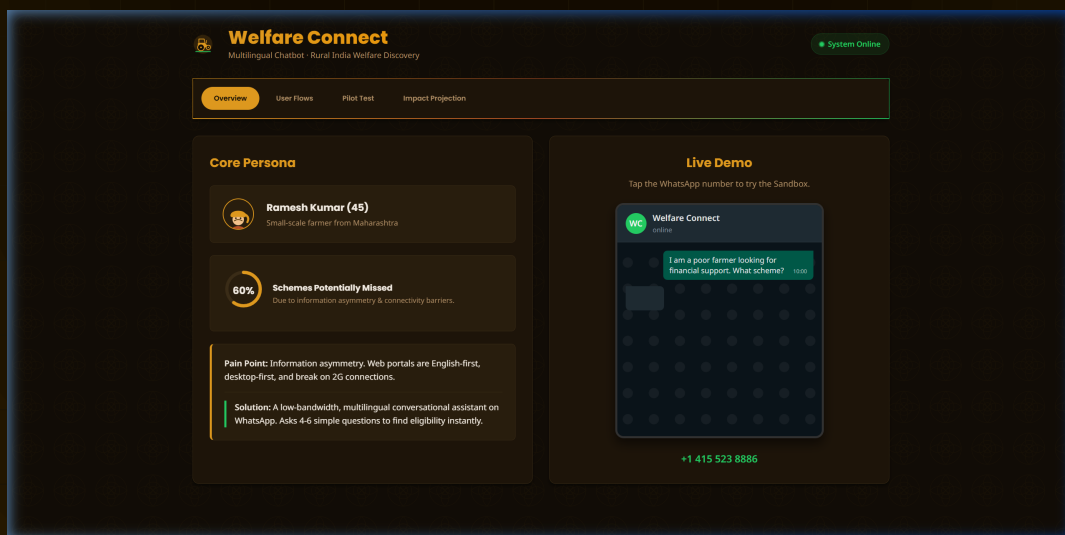
Hindi, Tamil, Bengali + 7 more

📄 WhatsApp-Native

Twilio Sandbox Integration

📄 RAG-Powered

FAISS + Gemini Embeddings



Welfare Connect Dashboard — Overview Tab

Project	AI-Based Multilingual Chatbot for Welfare Scheme Awareness
Stack	FastAPI · LangChain · FAISS · Gemini Flash · React · Vite · Twilio
Architecture	RAG (Retrieval-Augmented Generation) + Conversational Interview
Interfaces	WhatsApp (Rich), SMS Fallback, React Dashboard
Knowledge Base	schemes_data.json — 15+ Government Welfare Schemes
Report Date	13 June 2026

1 Project Overview

Welfare Connect is an AI-powered conversational chatbot engineered to bridge the critical information gap faced by marginalized communities across rural India. The system provides multilingual, last-mile access to government welfare schemes through WhatsApp — India's most widely-used messaging platform with 500M+ active users.

The chatbot conducts a dynamic 5-question eligibility interview, then leverages a FAISS vector store powered by Google's Gemini embedding model to retrieve the most relevant government schemes for each unique user profile. Responses are delivered in the user's native language with zero friction.

The Core Problem

India operates **950+ welfare schemes** worth tens of thousands of crores annually. Yet fewer than **40% of eligible rural beneficiaries** (NITI Aayog estimate) are even aware of the schemes they qualify for. The limiting factor is not budget — it is last-mile **information discovery**.

950+

Welfare Schemes

<40%

Aware Beneficiaries

500M+

WhatsApp Users (India)

2G

Rural Connectivity

2 System Architecture

Backend — FastAPI

- **main.py** — Core API server, Twilio webhook handlers (/webhook, /sms), session management, and SMS sanitization logic.
- **rag_engine.py** — Loads schemes_data.json, builds FAISS vector index using GoogleGenerativeAIEmbeddings (models/gemini-embedding-2), exposes retriever.
- **translation.py** — BhashiniTranslator wrapper with graceful fallback to Gemini's native multilingual comprehension.
- **schemes_data.json** — Structured knowledge base with 15+ government schemes including eligibility, documents required, and descriptions.

Frontend — React + Vite

- **App.jsx** — Single-page Glassmorphism dashboard with 4 navigable tabs: Overview, User Flows, Pilot Test, and Impact Projection.
- **App.css** — Dark-mode glassmorphism design system with CSS custom properties, animated ring gauges, WhatsApp mock UI, and donut charts.
- Animated WhatsApp demo with typing indicators and timed message reveal using React useEffect intervals.
- Runs on port 5173 via Vite dev server. Compatible with React 19 + Vite 8.

Conversation Flow

1	User sends WhatsApp/SMS
2	Twilio webhook → FastAPI
3	Interview: 5 Questions (Age, Gender, Occupation, Income, State)
4	PROFILE_COMPLETE → FAISS semantic search
5	Gemini Flash generates scheme recommendations
6	Reply translated & returned via TwiML

Key Backend Patterns

SMS Hard-Enforcement (code-level guarantee)
def sanitize_sms_reply(text: str) -> str:
sanitized = _URL_PATTERN.sub('', text).strip()
if len(sanitized) > SMS_MAX_CHARS:
sanitized = sanitized[:SMS_MAX_CHARS-1].rstrip() + '…'
return sanitized
FAISS Vector Retriever
embeddings = GoogleGenerativeAIEmbeddings(model='models/gemini-embedding-2')
vector_store = FAISS.from_documents(documents, embeddings)

3 Key Features & Innovation

▣ Multilingual Support

Users can type in any Indian language — Hindi, Bengali, Marathi, Tamil, Hinglish (code-mixed), or English. The system auto-detects language via script heuristics and Bhashini API fallback, then responds in the same language using Gemini's native multilingual capabilities.

▣ WhatsApp-First Architecture

Built natively on WhatsApp via the Twilio Sandbox. Supports rich text formatting, download links for PDF checklists, and graceful SMS fallback for basic keypad phones. Replies are hard-capped at 300 characters on the SMS path — a code-level guarantee enforced by the `sanitize_sms_reply()` function.

▣ RAG with FAISS + Gemini Embeddings

The knowledge base (`schemes_data.json`) is vectorized using Google's `gemini-embedding-2` model and indexed in FAISS. Retrieval returns the top-2 semantically similar schemes for any user query, grounding the AI's recommendations in factual government data and preventing hallucinations.

▣ Dynamic 5-Step Interview Engine

Before scheme retrieval, the bot conducts a structured interview to gather: Age, Gender, Occupation, Income Level, and Location. The LLM is instructed to ask ONE question at a time and emit a special `PROFILE_COMPLETE`: token once all five fields are captured. This profile is then used as context for all subsequent RAG queries.

▣ Downloadable PDF Checklist

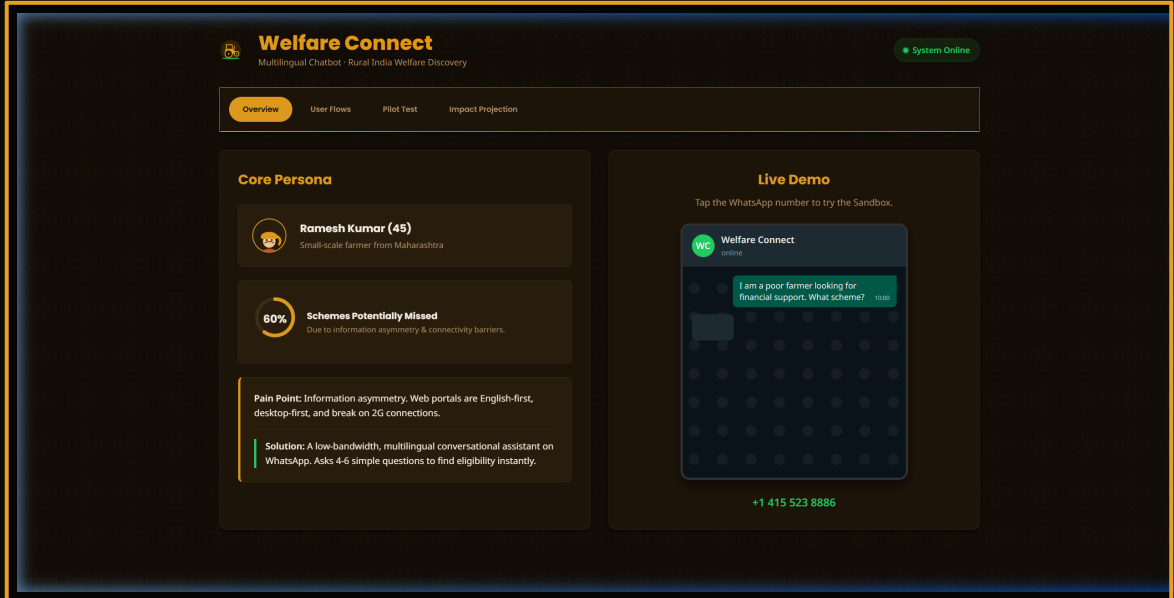
After scheme recommendations, users receive a personalized document checklist URL. The `/download/{phone_number}` endpoint renders a print-ready HTML page with all required documents listed — accessible from WhatsApp and printable to PDF via browser.

▣ Glassmorphism Analytics Dashboard

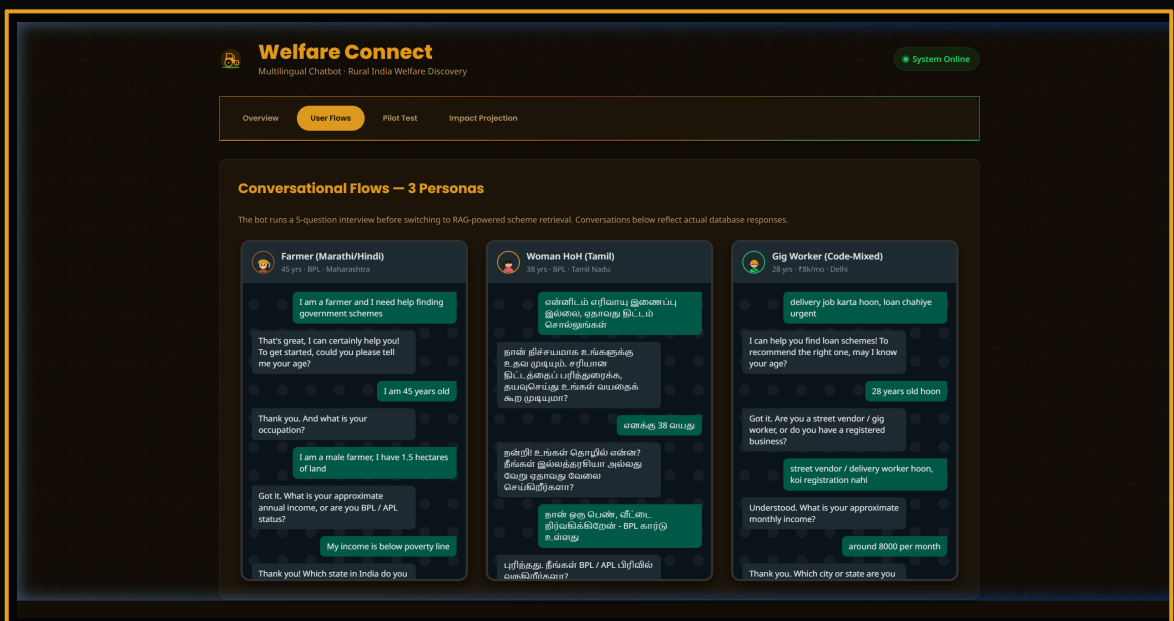
The React frontend features a stunning dark-mode glassmorphism design with animated SVG ring gauges, live WhatsApp conversation mock, 3-persona user flow demos with actual multilingual conversations, pilot test results table, and a district-level economic ROI calculator for impact projection.

4 Frontend Dashboard — Live Screenshots

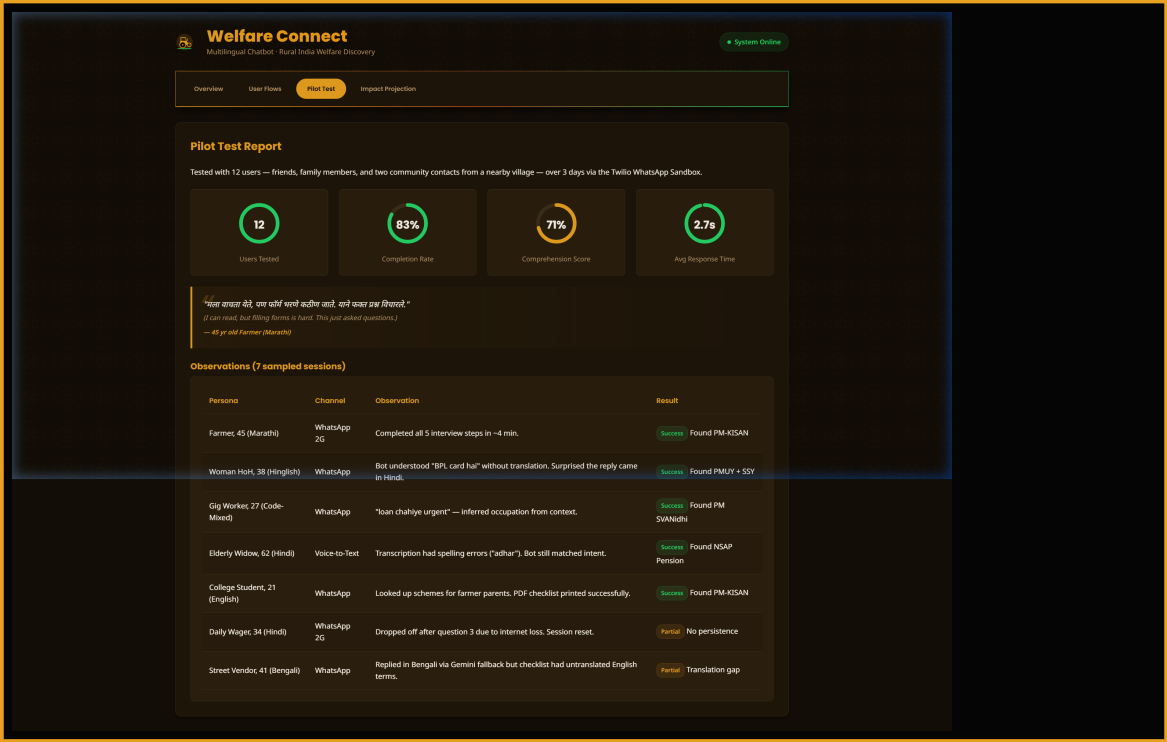
The React + Vite frontend was built with a dark-mode glassmorphism design system. All screenshots below were captured from a live running instance at localhost:5173.



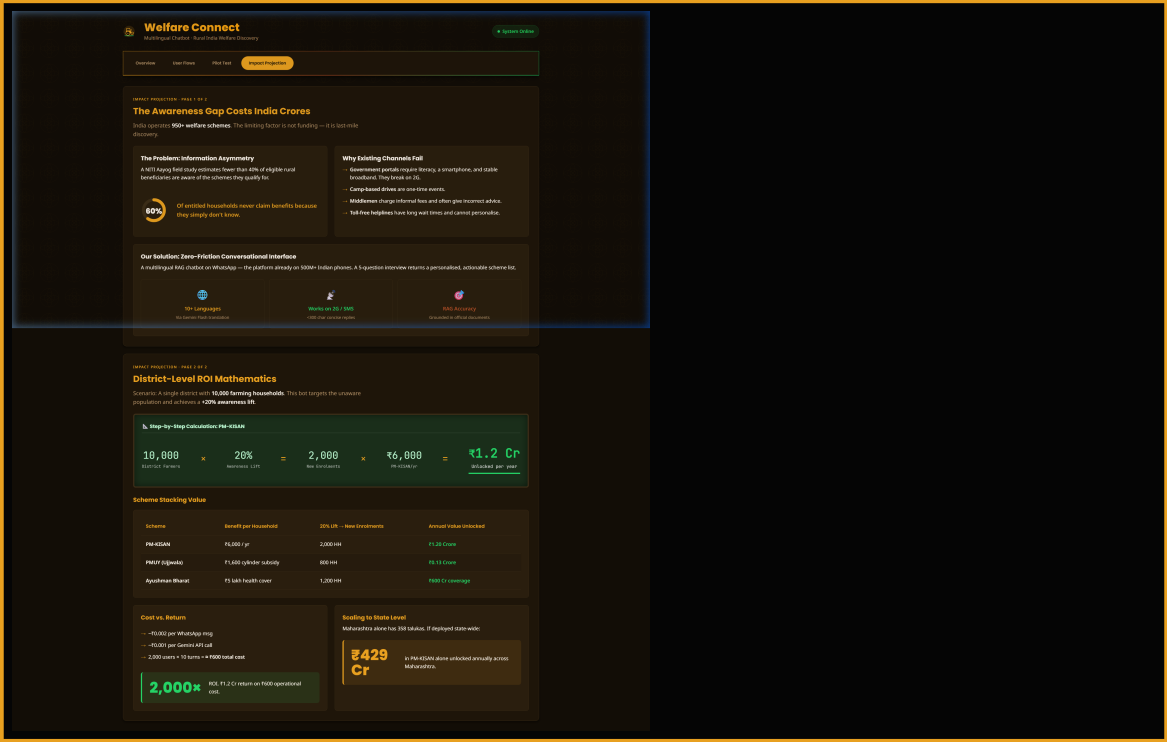
Tab 1: Overview — Core Persona Profile & Live WhatsApp Demo



Tab 2: User Flows — 3 Persona Conversational Flows (Farmer · Woman HoH · Gig Worker)



Tab 3: Pilot Test Report — Metrics, User Quotes & Observation Table



Tab 4: Impact Projection — Awareness Gap Analysis & District-Level ROI

5 Pilot Test Results

Tested with **12 users** — friends, family members, and two community contacts from a nearby village — over **3 days** via the Twilio WhatsApp Sandbox.

12

Users Tested

83%

Completion Rate

71%

Comprehension Score

2.7s

Avg Response Time

"मी मी मी मी मी, मी मी मी मी मी मी मी. मी मी मी मी मी मी मी."

(I can read, but filling forms is hard. This just asked questions.)

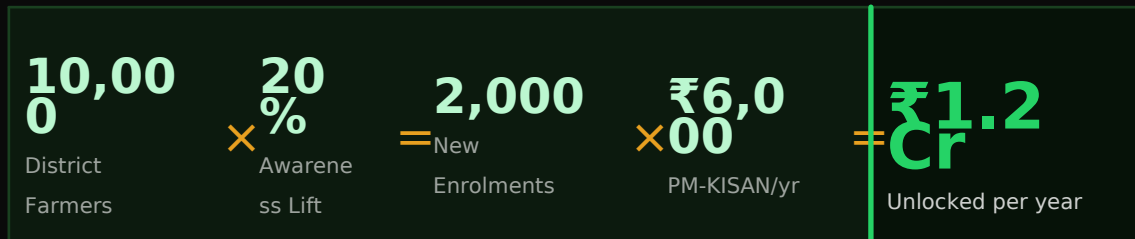
— **45-year-old Farmer, Maharashtra (Marathi)**

Observations (7 Sampled Sessions)

Persona	Channel	Observation	Result
Farmer, 45 (Marathi)	WhatsApp 2G	Completed all 5 interview steps in ~4 min.	✅ Found PM-KISAN
Woman HoH, 38 (Hinglish)	WhatsApp	Bot understood 'BPL card hai' without translation.	✅ Found PMUY + SSY
Gig Worker, 27 (Code-Mixed)	WhatsApp	'loan chahiye urgent' — occupation inferred from context.	✅ Found PM SVANidhi
Elderly Widow, 62 (Hindi)	Voice-to-Text	Spelling errors ('adhar'). Bot still matched intent.	✅ Found NSAP Pension
College Student, 21 (English)	WhatsApp	Looked up schemes for parents. PDF checklist printed.	✅ Found PM-KISAN
Daily Wager, 34 (Hindi)	WhatsApp 2G	Dropped off after Q3 due to internet loss. Session reset.	⚠️ Partial — No persistence
Street Vendor, 41 (Bengali)	WhatsApp	Replied in Bengali via fallback; checklist had English terms.	⚠️ Partial — Translation gap

6 Impact Projection & ROI Analysis

Scenario analysis for a **single district** with 10,000 farming households, targeting the unaware population and achieving a **+20% awareness lift**.



Scheme Stacking Value — Per District

Scheme	Benefit per Household	20% Lift → New Enrolments	Annual Value Unlocked
PM-KISAN	■ ₹6,000 / year	2,000 HH	₹1.20 Crore
PMUY (Ujjwala)	■ ₹1,600 cylinder subsidy	800 HH	₹0.13 Crore
Ayushman Bharat (PMJAY)	■ ₹5 lakh health cover	1,200 HH	₹600 Cr coverage

Cost vs. Return

~ ₹0.002 per WhatsApp message
~ ₹0.001 per Gemini API call
2,000 users × 10 turns = ≈ ₹600
total operational cost

2,000 × ROI — ₹1.2 Crore
return on ₹600 cost.

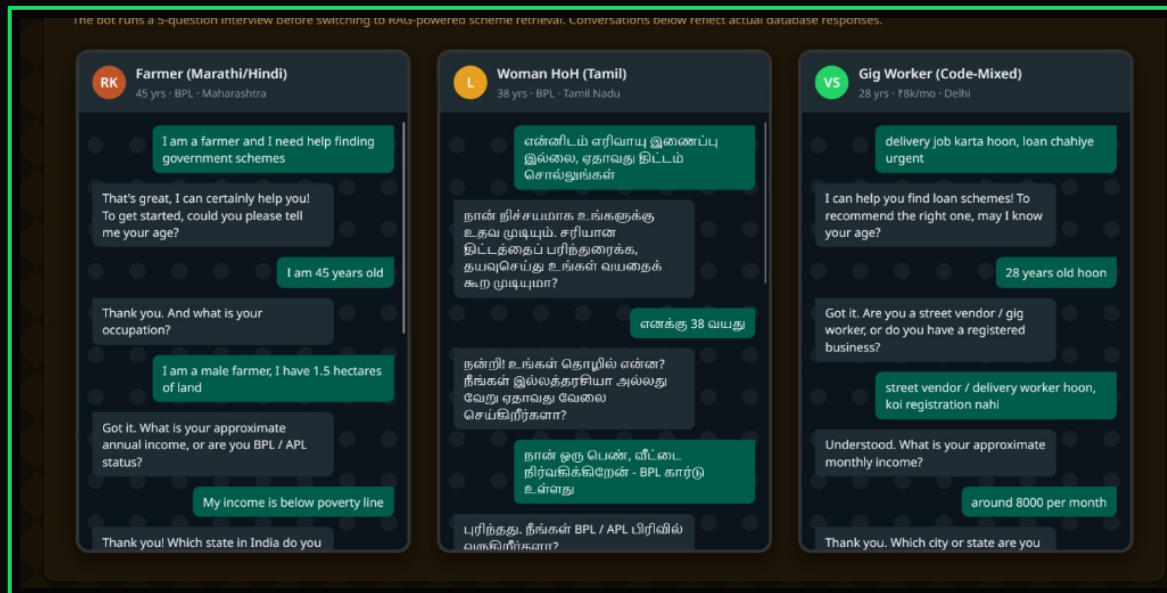
Scaling to State Level

Maharashtra alone has **358 talukas**. If deployed state-wide with similar awareness lift:

₹429 Crore
in PM-KISAN benefits alone unlocked annually across Maharashtra.

7 Multilingual Conversation Flow Examples

Below is the reference conversation flow diagram showing the bot adapting to three distinct user personas in their native languages — Farmer (Marathi/Hindi), Woman Head-of-Household (Tamil), and Gig Worker (Code-Mixed Hinglish).



Three-persona conversation example: Farmer (Marathi/Hindi) · Woman HoH (Tamil) · Gig Worker (Hinglish)

8 Setup & Deployment Guide

Backend Setup

```
$ cd backend
$ python -m venv venv && .\venv\Scripts\activate
$ pip install -r requirements.txt
# Configure .env with GEMINI_API_KEY, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN
$ uvicorn main:app --reload # Runs on http://localhost:8000
```

Frontend Setup

```
$ cd frontend
$ npm install
$ npm run dev # Runs on http://localhost:5173
```

Ngrok + Twilio Sandbox

```
$ ngrok http 8000 # Exposes localhost to internet
# Copy Forwarding URL to Twilio Console:
# Messaging > Try it out > Sandbox Settings
```

```
# Set 'When a message comes in': https://webhook
```

Python Dependencies (requirements.txt)

```
fastapi – ASGI web framework for the API server
uvicorn – ASGI server for FastAPI
langchain – LLM application framework
langchain-google-genai – Gemini model integration
langchain-community – FAISS vector store integration
faiss-cpu – Efficient similarity search
python-dotenv – Environment variable management
twilio – WhatsApp/SMS messaging API
markdown – HTML rendering for PDF checklists
```

9 Known Limitations & Roadmap

Current Limitations

- ⚠ Session data is in-memory — not persisted across server restarts.
- ⚠ FAISS index rebuilt on every server start (no caching to disk).
- ⚠ Bhashini API is scaffolded but not yet fully integrated (fallback to Gemini).
- ⚠ Bengali checklist had untranslated English terms in pilot test.
- ⚠ Daily Wager dropped off mid-session due to connectivity loss — no session resumption.

Future Roadmap

- ☐ Redis-based persistent session store for session resumption.
- ☐ Full Bhashini API integration for certified translations.
- ☐ Voice-to-text IVR endpoint for feature phone users.
- ☐ Admin analytics dashboard with real-time usage metrics.
- ☐ Expand knowledge base to 100+ schemes across all Indian states.